

Smart Recruitment Matching Platform

Full Technical Stack Document

Implementation-based technical stack summary prepared from the project repository and the approved Business Requirements Document (BRD).

Project Type	Secure web-based recruitment and job matching platform
Frontend	Angular 21.2.x, TypeScript, HTML, CSS, Tailwind CSS 4.x
Backend	ASP.NET Core Web API on .NET 8 using C#
Database	Microsoft SQL Server LocalDB (MSSQLLocalDB) via Entity Framework Core
Repository	AbiShayiny / Smart-Recruitment-Matching-Platform

Repository verification note: the repository contains separate Backend and Frontend projects. The integration branch was used for the implementation-level version details because it contains the Angular project files and the .NET project configuration.

1. Technical Stack Overview

The Smart Recruitment Matching Platform is structured as a client-server web application. The Angular frontend provides

role-based user interfaces for job seekers, employers, and administrators. The ASP.NET Core Web API provides authentication, user/profile, employer company, vacancy, matching, and administrative functionality. Entity Framework Core connects the backend to SQL Server LocalDB.

Layer	Technology / Component	Purpose
Presentation	Angular 21.2.x	Single-page web application and role-based UI
Frontend language	TypeScript, HTML, CSS	Application logic, templates and styling
UI styling	Tailwind CSS 4.x + CSS	Responsive layout and component styling
Backend	ASP.NET Core Web API (.NET 8)	REST-style HTTP API and server-side business logic
Backend language	C#	Controllers, services, repositories and matching logic
Data access	Entity Framework Core 8.x	ORM and SQL Server data access/migrations
Database	SQL Server LocalDB	Local relational database for development
Authentication	JWT Bearer	Token-based authentication and authorization

Password security	BCrypt.Net-Next	Password hashing and verification
API documentation	Swagger / OpenAPI via Swashbuckle	Interactive API documentation and testing

2. .NET Version and Backend Technology

The backend project targets .NET 8 and uses the Microsoft Web SDK. The project configuration explicitly targets net8.0. Entity Framework Core and the SQL Server provider are both on the 8.0.30 package line.

Component	Version / Technology
.NET target framework	net8.0 (.NET 8)
Web framework	ASP.NET Core Web API
Language	C#
ORM	Entity Framework Core 8.0.30
SQL Server provider	Microsoft.EntityFrameworkCore.SqlServer 8.0.30
EF design tools	Microsoft.EntityFrameworkCore.Design 8.0.30
EF tooling	Microsoft.EntityFrameworkCore.Tools 8.0.30
JWT authentication	Microsoft.AspNetCore.Authentication.JwtBearer 8.0.30
Password hashing	BCrypt.Net-Next 4.2.0
API documentation	Swashbuckle.AspNetCore 6.6.2

3. Frontend Technology / Framework

The frontend is an Angular application configured with Angular CLI/build tooling. The project uses TypeScript and Angular routing, with Tailwind CSS imported through the global stylesheet. The frontend is organized into feature areas such as authentication, job seeker, employer, jobs, matching, applications, contact requests, notifications, and administration.

Technology	Implementation detail
Framework	Angular 21.2.x
Angular CLI / build	@angular/cli 21.2.21; @angular/build 21.2.21
Language	TypeScript 5.9.x
HTML	Angular component templates
CSS	Global and component styling
Utility CSS	Tailwind CSS 4.1.x
Reactive library	RxJS ~7.8.0
Node package manager	npm 11.8.0
Testing	Vitest 4.0.8 + JSDOM 28.0.0
Formatting	Prettier 3.8.1
Build/PostCSS	PostCSS 8.5.3 + @tailwindcss/postcss 4.1.x

4. Database and Data Access

The repository configuration uses SQL Server LocalDB through the connection name DefaultConnection. The backend registers an

Entity Framework Core DbContext with UseSqlServer(), and the repository contains EF Core migration files for the application's relational schema.

- Database engine: Microsoft SQL Server LocalDB (MSSQLLocalDB).
- Database name configured for development: SMRPDb.
- Data access: Entity Framework Core with the SQL Server provider.
- Schema management: EF Core migrations are present in the Backend/Migrations folder.
- Database tool: SQL Server Management Studio (SSMS) can be used for database inspection and administration.

5. Important BRD vs Implementation Note

The BRD lists SQL Server Express as the proposed database technology. The repository configuration currently uses SQL Server LocalDB (MSSQLLocalDB) for development. For an academic technical-stack document, the implementation value should be reported as LocalDB, while the BRD proposal can be noted separately if required by the assessor.

6. Programming Languages

Language / Format	Where it is used
C#	ASP.NET Core backend, controllers, services, repositories, matching/business logic
TypeScript	Angular components, pages, services, guards, models and routing

HTML	Angular component templates and application structure
CSS	Global/component styling; Tailwind CSS integration
SQL	Relational database concepts and SQL Server/EF Core generated database operations
JSON	API payloads, application configuration and package metadata

7. APIs / Services Used

API / Service	Purpose
ASP.NET Core Web API	Server-side REST-style endpoints consumed by the frontend.
Authentication API	Registration and login endpoints; JWT token-based access.
Admin API	User management and dashboard-related endpoints.
Job Seeker API	Profile and CV-related operations.
Employer Company API	Company profile creation, retrieval and update.
Employer Vacancy API	Vacancy creation, retrieval, update and closing.

Matching service	Application/backend matching structure for comparing candidate and vacancy information.
Swagger / OpenAPI	Interactive API documentation/testing during development.
CORS	Allows the Angular development origin (localhost:4200) to call the backend.
External third-party APIs	No external business API integration is evidenced in the inspected repository.

8. Application Architecture and Technical Components

The backend follows a layered organization visible in the repository: Controllers expose HTTP endpoints, Services contain application/business operations, Repositories provide data-access abstractions, Models represent persisted entities, DTOs define API data contracts, and Data contains the EF Core DbContext. The frontend is organized by feature modules/pages with shared components, guards, interceptors, models and services.

- Frontend feature areas: authentication, administrator, employer, job seeker, jobs, matching, applications, contact requests and notifications.
- Backend areas: authentication, administration, job seeker, employer/company, vacancy, matching, data/repositories/services and CV storage.
- Security: JWT Bearer authentication, authorization middleware, BCrypt password hashing and CORS configuration.

- CV handling: the BRD requires protected storage outside the public web folder; the backend contains aStorage area for this purpose.
- API discovery: Swagger/OpenAPI is configured for development.

9. Other Tools, Libraries and Technologies

Tool / Library	Role in the project
Visual Studio / .NET tooling	Backend development, build and debugging
Visual Studio Code	Frontend development and Angular project work
Git	Source control and branch-based development
GitHub	Remote source-code repository and collaboration
SQL Server / LocalDB	Relational development database
SQL Server Management Studio (SSMS)	Database inspection and management
Entity Framework Core	ORM, LINQ-based data access and migrations
BCrypt.Net-Next	Password hashing
Swashbuckle.AspNetCore	Swagger/OpenAPI generation
JWT Bearer middleware	Authentication token validation
Angular Router	Client-side routing
RxJS	Reactive programming support in Angular
Tailwind CSS	Utility-first frontend styling
PostCSS	CSS processing pipeline
Vitest / JSDOM	Frontend unit-test environment
Prettier	Code formatting

npm	Frontend package management and scripts
-----	---

10. Security and Technical Practices

- JWT Bearer authentication is configured in the ASP.NET Core pipeline.
- Issuer, audience and token lifetime validation are enabled.
- Passwords are hashed and verified using BCrypt.Net-Next rather than stored as plain text.
- CORS is explicitly configured for the Angular development origin.
- Swagger is exposed under the development environment for API documentation/testing.
- The BRD specifies protected CV storage outside the public web folder; this is reflected by the backendStorage area and CV upload functionality.

11. BRD Technology Alignment

The approved BRD describes the intended architecture as a JavaScript/HTML/CSS frontend, ASP.NET Core Web API using C#, SQL Server, Entity Framework Core, JWT authentication, secure password hashing, C# rule-based matching, and protected CV storage. The repository implements these core technology choices, with Angular/TypeScript used as the concrete frontend framework.

BRD requirement	Implementation found
Frontend: JavaScript, HTML, CSS	Angular + TypeScript + HTML + CSS; Tailwind CSS is also configured.

Backend: ASP.NET Core Web API using C#	.NET 8 ASP.NET Core Web API with C#.
Database: SQL Server Express (proposed)	SQL Server LocalDB is configured in the current implementation.
ORM: Entity Framework Core preferred	EF Core SQL Server provider and migrations are present.
JWT authentication	JWT Bearer authentication is configured.
Secure password hashing	BCrypt.Net-Next is used for password hashing/verification.
Matching logic in C#	Matching service/controller structure exists in the project; exact scoring weights should be documented from the final matching implementation if required.
Protected CV storage	CV upload support and a backend Storage area are present; the BRD requires storage outside the public web folder.

12. Conclusion

The Smart Recruitment Matching Platform uses a modern web stack centered on Angular 21.2.x and ASP.NET Core/.NET 8, with C# and TypeScript as the primary application languages. SQL Server LocalDB and Entity Framework Core provide relational persistence, while JWT, BCrypt, Swagger/OpenAPI, Tailwind CSS,

RxJS and standard Angular tooling support security, development and UI delivery.

This document intentionally reports the implementation-level technologies visible in the repository. Where the BRD describes a proposed technology or a requirement not fully evidenced by the inspected code, it is identified separately rather than presented as a confirmed implementation.